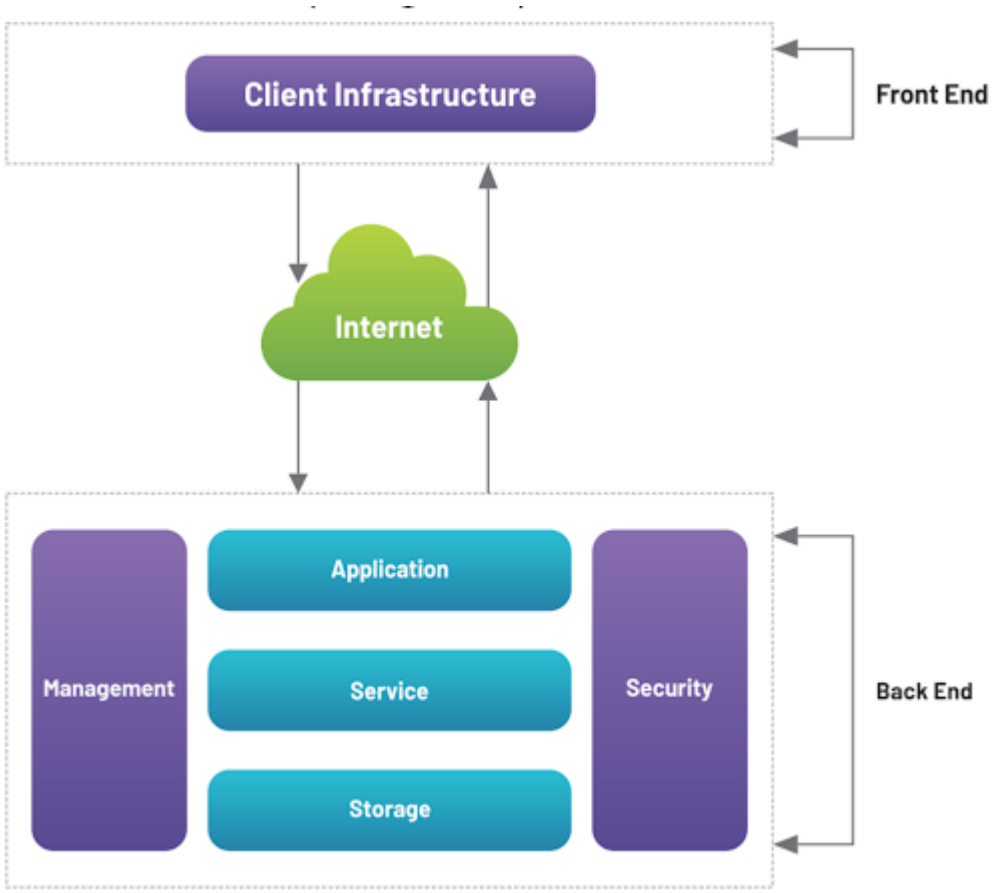
➢ Cloud storage system architecture

Cloud storage system architecture includes a front-end for user access, a back-end with virtualized servers and storage, and networking to connect them.

It consists of components like a storage resource pool, storage virtualization software, and access interfaces (like APIs or web services) that allow users to interact with the stored data on demand. The architecture supports different storage types (object, block, file) and provides features like scalability, elasticity, and multi-tenancy.



A traditional cloud framework involves a delivery model, a network, security and management layers, and most importantly, the back-end and front-end,

- **Front-end**. The collection of elements a client interacts with is the user interface and application a person uses to get to the cloud services.
- **Back-end**. The cloud that's leveraged by the cloud provider that holds, manages, and secures resources. In addition to this, it contains storage, virtual machines, ways to control traffic, deployment models, and so on.
- **Storage**: Virtual place to manage and safekeep data.
- **Management**: Elements that help take care of the application, service, runtime, storage, infrastructure, and other security mechanisms.
- **Security**: Practice and tools to employ diverse security mechanisms to keep cloud resources, systems, and files, safe for end users.
- **Network**: Internet connection that acts as a bridge between the front-end and the back-end, allowing them to talk and interact with each other.

➢ Virtual Data Center (VDC)

A virtual data center (VDC) is a cloud-based IT infrastructure that provides compute, storage, and networking resources on demand through virtualization, abstracting these from physical hardware. Its architecture includes virtual machines (VMs), virtual networking, and virtual storage, all managed by a hypervisor and orchestration software. Key components are hosted in physical data centers and delivered as a service, allowing organizations to scale resources as needed, simplify IT management, and reduce costs.

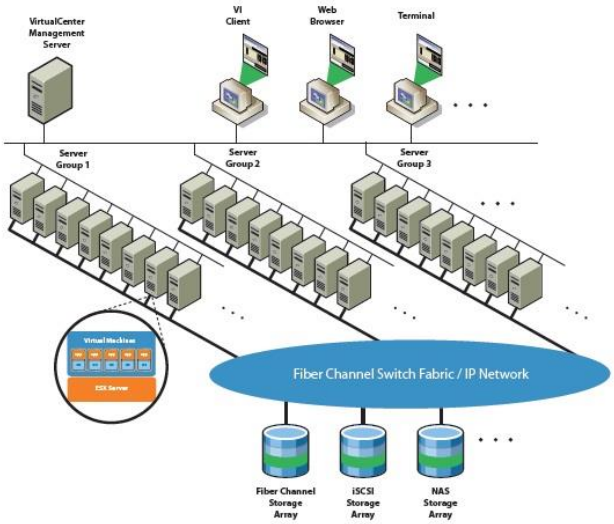
VDC Architecture and Environment

Architecture

- **Virtual Machines (VMs):** Software-based emulations of physical computers that run an operating system and applications on a hypervisor.
- **Hypervisor:** The foundational software layer that allows multiple VMs to run on a single physical server, optimizing hardware utilization.
- **Virtual Networking:** Software-defined virtual switches, routers, and load balancers that enable communication between VMs and external networks.
- **Virtual Storage:** A centralized pool of software-defined storage that can be flexibly allocated to VMs, supporting features like snapshots and replication.
- **Management and Orchestration:** Tools used to automate the provisioning, configuration, and monitoring of all VDC components.
- **Security:** Virtual security components like firewalls, intrusion detection systems, and access controls are used to protect the environment.

Environment

- **Physical Foundation:** A VDC runs on top of physical servers, storage, and networking equipment located in a cloud provider's data center.
- **Cloud-Based:** Resources are delivered as a service, eliminating the need for organizations to own and maintain physical hardware.
- **Scalability:** Resources can be scaled up or down dynamically based on demand, allowing for efficient use of capacity.
- **Accessibility:** VDCs can be managed and accessed remotely from anywhere, simplifying IT operations.



➢ VDC Techniques

- **Server Virtualization:** Creates multiple virtual servers (VMs) from a single physical server, allowing for better resource utilization and reduced hardware costs.
- **Storage Virtualization:** Abstracts the logical storage from the physical storage, enabling storage from multiple devices to be managed as a single pool.
- **Network Virtualization:** Logically separates network resources from the underlying physical hardware, allowing for the creation of virtual networks that can be segmented and managed independently for better security and efficiency.
- **Desktop Virtualization:** Separates the desktop operating system and applications from the end-user's physical hardware, allowing multiple virtual desktops to run on a central server.
- **Application Virtualization:** Encapsulates applications in a virtual environment, separating them from the underlying operating system and hardware. This allows for centralized management and deployment of applications without local installation.

➢ Benefits of a virtual data center architecture

- **Increased agility and flexibility:** Resources can be provisioned, scaled, and de-provisioned much faster than with physical hardware.
- **Reduced costs:** Lowers capital expenditures (CapEx) by reducing the need for physical hardware and operational expenses (OpEx) through lower power, cooling, and maintenance costs.
- **Improved resource utilization:** Consolidates workloads onto fewer physical servers, maximizing hardware efficiency.
- **Enhanced scalability:** Allows for easier scaling of resources up or down based on demand.
- **Simplified management:** Provides a centralized portal for managing all virtualized resources, including network and security settings.

➢ Google File System (GFS)

The Google File System (GFS) was a proprietary, scalable distributed file system developed by Google in the early 2000s to handle its immense data storage and processing needs. Designed to run on large clusters of inexpensive commodity hardware, GFS was optimized for Google's application workloads, which involved massive files, frequent appends, and high-throughput data access.

Core architectural components

A GFS cluster consists of a single master, multiple chunkservers, and multiple clients.

- **Master server:** The master is the central coordinator of the GFS cluster. It stores all the file system's metadata in memory, which includes:
  - The file and directory namespace.
  - The mapping of files to their chunks.
  - The locations of the chunk replicas on the chunkservers.
    To provide high availability and fast recovery, the master's state is backed up to an operation log and replicated on other machines.
- **Chunkservers:** These are the workhorses of the system, storing data as fixed-size chunks (typically 64 MB) on their local disks. A chunk is a plain Linux file on the chunkserver. Chunkservers communicate with the master via periodic heartbeat messages to report their status and the chunks they hold.
- **Clients:** GFS client code is linked into application programs to interact with the master and chunkservers. Clients communicate with the master for metadata-related requests but read and write file data directly to the chunkservers.

Google File System (GFS) features

- **Fault tolerance:** GFS is designed to handle failures by replicating data across multiple machines and by having a master node that can be quickly recovered or replicated.
- **Master-Chunkserver architecture:**
  - A **master node** manages all file system metadata, such as namespaces, file-to-chunk mapping, and chunk locations.
  - Multiple **chunkservers** store the actual data in fixed-size chunks (typically 64 MB) and handle read/write requests.
- **Large file support:** The system is optimized for large files (≥ 100 MB) and operations on them, which is ideal for batch workloads like web crawling and indexing.
- **Optimized for appends:** GFS prioritizes appending data to the end of files over overwriting existing data, reflecting common Google workloads.
- **High throughput and scalability:** It is designed to deliver high aggregate performance by distributing data and requests across thousands of machines and petabytes of storage.
- **Atomic record appends:** GFS provides a mechanism for multiple clients to append data to the same file concurrently and atomically.
- **Snapshotting:** GFS includes a snapshot feature that allows for creating quick backups of file system state.
- **Fast recovery:** Both the master and chunkserver processes are designed for rapid startup in case of a crash or restart.
- **In-memory metadata:** The master stores critical metadata in memory for fast access, though it is also logged to disk for persistence and replication.

Advantages

- **Scalability:** GFS is designed to scale to petabytes of data by distributing it across thousands of servers.
- **Fault Tolerance:** It achieves high reliability through data replication across multiple chunk servers, which is critical since component failures are common in large-scale systems.
- **Cost-Effectiveness:** By using inexpensive commodity hardware, GFS is more cost-effective than solutions requiring specialized hardware.
- **Performance for Specific Workloads:** It is highly optimized for sequential reads and appends, which aligns with Google's original use case of indexing the web.
- **High Availability:** The system is designed to handle individual server failures gracefully, ensuring high availability.
- **Reduced Master Interaction:** Clients cache chunk locations, minimizing their need to interact with the master for every operation, which reduces network overhead.

Disadvantages

- **Single Point of Failure:** The single master server, while simplifying the design, can become a bottleneck and a single point of failure if it fails.
- **Poor for Small Files:** GFS is not optimized for small files because its large chunk size can lead to inefficiency, with a small file potentially occupying a whole chunk.
- **Limited Random Access:** The system performs poorly with workloads that require random access writes, as it is designed for sequential I/O.
- **Consistency Model:** It uses a relaxed consistency model, meaning clients must perform their own consistency checks, which adds complexity.
- **Higher Storage Overhead:** Replication for redundancy means GFS consumes more raw storage than some other distributed file systems.

➢ Hadoop Distributed File System (HDFS)

The Hadoop Distributed File System (HDFS) is a Java-based, distributed file system designed to store and manage large datasets across clusters of commodity servers, providing fault tolerance, high availability, and high-throughput access for big data processing. It works by breaking data into smaller blocks, distributing these blocks across multiple DataNodes (slave nodes) in the cluster, and managing these files and their locations via a master NameNode (master node).

Terminology

- **NameNode:**

  The master node that manages the file system namespace, including file and directory metadata (filenames, permissions, block locations, etc.). There is typically one active NameNode and an optional Standby NameNode for high availability.

- **DataNode:**

  The slave nodes that store the actual data blocks. They handle read/write requests from clients and perform block creation, deletion, and replication as instructed by the NameNode.

- **Block:**

  The smallest unit of data that HDFS stores. Files are broken down into blocks, which are then distributed across DataNodes. The default block size is typically 128MB or 256MB.

- **Replication Factor:**

  The number of copies of each data block stored across different DataNodes to ensure fault tolerance and data availability. The default replication factor is usually three.

- **Client:**

  The application or user interacting with HDFS to read or write data.

- **Rack:**
  A physical grouping of DataNodes in a data center. HDFS employs a rack-aware replica placement policy to improve fault tolerance and network performance.

Features

- **Distributed Storage:**

  Stores large files by splitting them into blocks and distributing these blocks across multiple DataNodes in a cluster.

- **Fault Tolerance:**

  Achieved through data replication, ensuring data availability even if some DataNodes fail.

- **Scalability:**

  Can be scaled horizontally by adding more DataNodes to the cluster as data storage requirements grow.

- **High Throughput:**

  Optimized for batch processing of large datasets, providing high throughput for data access and transfer.

- **Cost-Effectiveness:**

  Designed to run on inexpensive, commodity hardware, making it a cost-effective solution for large-scale data storage.

- **Data Locality:**

  Attempts to move computation closer to the data (e.g., running MapReduce tasks on DataNodes that store the relevant data blocks) to minimize network traffic.

- **Write Once, Read Many:**
  Optimized for workloads where files are written once and then read multiple times, making it suitable for analytical applications.

Limitations

- **Low-Latency Data Access:**

  Not suitable for applications requiring low-latency data access (e.g., interactive queries or transactional systems) due to its design for high-throughput batch processing.

- **Small File Inefficiency:**

  Storing a large number of small files can be inefficient as each file, regardless of size, incurs overhead in the NameNode's memory for metadata management.

- **No Random Writes/Appends:**

  HDFS primarily supports appending data to existing files but does not efficiently support random writes or modifications within a file once it has been written.

- **Single NameNode Bottleneck (Traditional HDFS):**
  In older versions, a single NameNode could become a bottleneck for very large clusters or high metadata operation workloads. This has been addressed with High Availability (HA) features.

➢ An architecture of federated Cloud computing

In a federated cloud architecture, multiple autonomous cloud service providers collaborate to offer a unified cloud experience to users by sharing resources and interoperating their systems. Key components include the Cloud Broker, acting as an intermediary for services and pricing, and Users, who access these services through the broker or directly. The model relies on resource sharing and interoperability to achieve scalability, availability, and cost optimization for services like applications, platforms, and infrastructure.

Model and Components

A typical federated cloud architecture includes the following key components:

Service Providers: Multiple independent cloud providers that make their infrastructure, platforms, and services available for federation.

Users: The end-users, who could be individuals, organizations, or applications that consume the services offered by the federated cloud.

Cloud Broker: A crucial intermediary that helps users find, select, and integrate services from different providers, ensuring they get the best value and meet their specific needs.

Cloud Exchange: A mechanism that maps user demands to the services offered by different providers within the federation.

Cloud Coordinator: A component responsible for assigning resources from various providers to meet user demands and manage the overall workload.

Explanation of the Model

1. Autonomy and Interconnection: Each cloud service provider in the federation remains an independent and autonomous entity but establishes connections with other providers to share resources.

2. Unified Service Delivery: The goal is to present a single, unified cloud environment to the users, abstracting away the complexities of interacting with multiple individual providers.

3. Interoperability: The providers' systems must be able to interoperate, allowing for smooth communication and resource sharing between their diverse infrastructures.

4. Location Transparency: Applications running in a federated cloud should be location-agnostic, meaning they don't need to know the specific cloud provider where their components are running.

5. Resource Sharing and Demand Accommodation: The federation allows providers to share resources to accommodate spikes in demand and to offer a broader range of services than any single provider could offer alone.

6. User Interaction: Users can interact with the federated cloud, typically through a cloud broker, to discover and access services based on criteria like cost, performance, and trust.

7. Monitoring and SLA Management: Continuous monitoring is essential to track service performance against Service Level Agreements (SLAs), with mechanisms in place to detect and address SLA violations.

In essence, federated cloud computing creates a "cloud of clouds" where providers collaborate while maintaining their independence, offering users increased availability, scalability, and optimized costs through resource pooling and shared management.

➢ Service Level Agreement (SLA)

A Service Level Agreement (SLA) is a formal contract between a service provider and a customer that defines the specific services to be delivered, the agreed-upon performance metrics, and the consequences for failing to meet those standards. It clarifies expectations, sets measurable goals, and establishes accountability for both parties, ensuring transparency and providing security for the customer while guiding the provider.

Key Components of an SLA

Scope of Service: Clearly outlines the services the provider is responsible for delivering.

Performance Metrics: Details the key performance indicators (KPIs) by which the service's quality and performance will be measured, such as uptime, response times, and resolution times.

Responsibilities: Specifies the obligations of both the service provider and the customer.

Service Level Objectives (SLOs): Measurable targets that define the desired level of service quality and performance.

Remedies and Penalties: Explains what happens if the agreed-upon service levels are not met, which could include financial penalties or other remedies.

➢ Why SLAS Are Important

Establishes Clear Expectations: An SLA creates a shared understanding between the provider and customer, minimizing misunderstandings.

Ensures Accountability: It holds the service provider accountable for delivering the promised level of service.

Provides Transparency: For the customer, an SLA offers transparency regarding the services they are receiving and the standards they can expect.

Protects Both Parties: An SLA protects both the customer and the provider by documenting terms and preventing claims of ignorance if issues arise.

Guides Service Provision: It serves as a concrete guideline for the service provider, clearly delineating their responsibilities and boundaries.

➢ SLA management:

SLA management involves five phases:
Feasibility (assessing the viability of an SLA),
On-Boarding (defining and formalizing the SLA),
Pre-production (preparing for the service delivery),
Production (the actual service delivery and monitoring), and
Termination (ending the agreement).
Each phase builds on the success of the previous one, ensuring the service level agreement is properly established, maintained, and concluded to meet both the service provider's and the client's needs.

1. Feasibility

- **Purpose**: To determine if hosting an application on a specific platform is technically, financially, and strategically viable.
- **Activities**: Assessing the required infrastructure, potential costs, and overall benefits.

2. Onboarding

- **Purpose**: To move the application to the hosting platform and perform initial performance analysis.
- **Activities**: Migrating the application, setting up basic configurations, and conducting early performance checks.

3. Pre-production

- **Purpose**: To thoroughly test and validate the SLA before the application goes live.
- **Activities**: Running the application in a dedicated test environment to ensure it meets all performance metrics and requirements outlined in the SLA.

4. Production

- **Purpose**: To launch the application and begin managing it under the terms of the SLA.
- **Activities**: The application goes live for end-users while performance is continuously monitored against the agreed-upon service level targets.

5. Termination

- **Purpose**: To officially end the service and the associated SLA.
- **Activities**: De-commissioning the service, archiving data, and closing out the contract.

➢ Types of SLA: Infrastructure SLA and Application SLA

Infrastructure and Application SLAs are two distinct types of Service Level Agreements that focus on different layers of a technology stack. The former covers the underlying hardware and network, while the latter addresses the performance and availability of the software running on top of that infrastructure.

➢ Infrastructure SLA

An Infrastructure SLA defines the quality of service for the fundamental components that make up a system. It ensures that the core hardware and network are consistently available and perform as expected.

Key metrics typically included in an Infrastructure SLA:

Availability/Uptime: The percentage of time a system, such as a server or network, is available and operational. For example, a 99.9% uptime guarantee for servers.

Response time/Latency: The amount of time it takes for a request to travel between a client and the server.

Packet delivery: The percentage of data packets that are successfully received compared to the total number sent.

Power and environmental conditions: In a data center setting, this covers the maintenance of power supplies, climate control, and physical security.

Disaster recovery: Specifies the time it will take for a service to be restored after a major outage (Mean Time to Recovery).

➢ Application SLA

An Application SLA focuses on the performance and availability of a specific software application or service from the user's perspective. It measures the quality of the end-user experience, which depends on both the application itself and the underlying infrastructure.

Key metrics typically included in an Application SLA:

First-call resolution rate: The percentage of customer issues resolved during their first interaction with a support team.

Error rate: The frequency of errors that occur within the application, such as failed transactions or coding errors.

Transaction time: The time it takes for a specific user action or transaction to be completed within the application.

Resolution time: The average time it takes for an application-related issue, once logged, to be completely fixed.

➢ Life cycle of SLA:

1. Contract Definition

This initial phase involves the service provider formally outlining its service offerings and the corresponding baseline SLAs.

- **Provider actions**: Standardized service catalogs and SLA templates are created for a variety of potential customer needs.
- **Template customization**: These standardized offerings can be customized to create unique SLAs for individual enterprise customers.

2. Publishing and Discovery

Once the contract templates and service offerings are defined, they are advertised so potential customers can find and review them.

- **Advertising**: The service provider advertises its service catalog through various standard publication media.
- **Discovery**: Customers can then search this catalog and compare offerings from multiple providers to find one that best fits their requirements for further negotiation.

3. Negotiation

This phase involves the service provider and the customer mutually agreeing on the terms and conditions of the SLA before signing the final agreement.

- **Customization**: For customized applications, this is a manual process where the provider analyzes the application's behavior to set specific performance and scalability metrics.
- **Automation**: For standard, packaged service offerings, this phase is often automated.
- **Finalization**: The negotiation ends when both parties have mutually agreed upon and signed the final SLA.

4. Operationalization

After the SLA is signed, it is operationalized. This involves actively managing, monitoring, and enforcing the terms of the agreement throughout its active lifespan.

- **Monitoring**: Service parameters defined in the SLA are measured and tracked to detect any deviations.
- **Accounting**: A record of SLA adherence is kept and archived for compliance purposes.
- **Enforcement**: Corrective actions are taken when monitoring detects a breach or violation of the SLA.

5. De-commissioning

The final phase occurs when the service agreement and the hosting relationship between the provider and consumer come to an end.

- **Termination**: This involves terminating all activities under the specific SLA.
- **Archival**: The final agreement is legally terminated and archived, which is an important step for record-keeping and compliance.

➢ Cloud Service Life Cycle

The cloud service life cycle includes **planning**, **creation** (design/transition), **operation**, and **termination** (decommissioning). This cycle, often framed by the ITIL framework, involves strategic planning, designing the service, transitioning it to the cloud environment, operating it, and continuously improving it until it is eventually terminated.

1. Planning (Service Strategy)

- **Define service strategy:** Identify and evaluate potential cloud services based on market demand, business goals, and competition.
- **Build a roadmap:** Create a plan and strategy for building and implementing the cloud service.

2. Creation (Service Design & Transition)

- **Service Design:** Determine the technical and functional requirements, necessary resources, and architecture for the service.
- **Service Transition:** Deploy the service into the cloud environment, which includes testing to ensure it meets security and quality standards before it is made available to users.

3. Operation (Service Operation)

- **Deliver and maintain:** Provide the service to users, monitor its performance, and perform maintenance as needed.
- **Manage incidents:** Provide customer support and manage service disruptions and other incidents.

4. Termination (Decommissioning)

- **Retirement:** Once the service is no longer needed, it must be retired and decommissioned.
- **Archiving/Destruction:** This involves the proper archiving or destruction of data and resources according to security and compliance policies.